

CROSS-LIB AND CROSS-LIB-HD

<http://www.github.com/Fabrizio-Caruso/CROSS-LIB/>

by Fabrizio Caruso

INTRODUCTION

The goal of this article is to present Cross-Lib and Cross-Lib-HD, briefly show how to use them and present 9 "universal" games I have written with it. By "universal" here I mean that these games will run on more than 200 different systems (consoles, computers, arcade boards, pocket calculators, etc.) including obviously the Oric 1 and Oric Atmos despite using the very same game code for all targets. The article also provides some details on the recently improved support for the Oric 1 and Oric Atmos.

WHAT IS CROSS-LIB ?

Cross-Lib (<https://github.com/Fabrizio-Caruso/CROSS-LIB/>) is a free open source framework for retro-coding for a multitude of 8-bit machines and other vintage systems from the '80s (computers, consoles, pocket calculators, arcade boards, etc.) such as Apple II, all 8-bit Commodore computers, Atari 800, Amstrad CPC, Sinclair ZX 81, Sinclair ZX Spectrum, TRS-80 CoCo, Dragon 32, TRS-80 MC-10, Thomson Mo5 and To7, MSX, Oric-1, Oric Atmos, BBC Micro, etc. as well as consoles such as GameBoy, Sega Master System, GameGear, Nintendo NES, etc. and other devices such as Texas Instruments pocket calculators. The supported CPU architectures include Zilog 80, MOS 6502, Intel 8080, Intel 8088/8086, Motorola 6803, Motorola 6809, RCA 1802, TMS9900, etc. Cross-Lib supports several lesser known systems such as the Philips VG-5000, the Jupiter Ace and several exotic and rare systems such as the Yugoslav Galaksija, the Hungarian Homlab-2, several computers from the Eastern German Robotron series such as the Robotron KC 85, etc. A large, yet partial list of the supported systems is at the page: <https://github.com/Fabrizio-Caruso/CROSS-LIB/blob/master/docs/STATUS.md>.

The main goal of Cross-Lib is to provide a *hardware abstraction layer* for hundreds of vintage systems so that the *very same* game code and the *very same* graphics assets can be used without any adaptation for all its supported systems.

Several articles have been published on Cross-Lib ([rg], [prog], [app2]) and its games ([ceo], [app]).

CROSS-LIB-HD

Cross-Lib-HD (<https://github.com/Fabrizio-Caruso/CROSS-LIB-HD/>) is just Cross-Lib with more graphics tiles at the cost of slightly bigger binaries and of fewer colors for a very limited set of targets. Cross-Lib-HD is fully backwards compatible with Cross-Lib, i.e., any project written for Cross-Lib will be compatible with Cross-Lib-HD. So any Cross-Lib project can be easily extended in Cross-Lib-HD with no need to rewrite the whole code. Cross-Lib-HD is not a complete replacement for Cross-Lib as it can produce worse results for small games for very memory-limited targets with limited tiles and/or color palettes. Depending on memory and color constraints, the user may choose one or the other or both if they want to build a standard version of a game together with an enhanced version (that may only work on targets with expanded memory and maybe with fewer colors in some cases). In the future Cross-Lib-HD will be integrated in Cross-Lib so that a user can decide which one to use for its game. The game code may use conditional macros and have extra tiles when compiled with Cross-Lib-HD, and still work with Cross-Lib.

There are currently no other differences other than the number of tiles between the two versions. For this reason from now on in this article we will refer collectively to both of them as Cross-Lib.

A LIBRARY AND A SCRIPT

Cross-Lib has two main components:

1. A C library (e.g., a collection of C APIs mostly for input/output), which provides a hardware abstraction layer;
2. The x1 Python script to manage game projects (e.g., to build, create, clone, etc.) and manipulate graphics assets (e.g., to create, import or rip assets)

CROSS-LIB VS OTHER DEV-KITS

Historically, 8-bit systems were programmed directly on the system and mostly in Assembly, or for simpler programs in interpreted BASIC.

Today cross-development simplifies developers' tasks a lot (modern editors, simplified debugging, efficient and quick compilation, etc.). Moreover cross-development can be done in C.

STANDARD C CODE IS NOT TOTALLY PORTABLE

The problem with the portability of C code comes from the diversity of the machines and from the fact that ANSI C does not provide a standard library for graphics, sounds, input, etc. A partial solution is already provided by some dev-kits such as CC65 (for MOS 6502), Z88DK (for the Zilog 80 and Intel 8080), LCC1802 (for the RCA 1802), CMOG (for the Motorola 6890), etc. By using one of these dev-kits, we can exploit their cross-compatible libraries for input and output. On the other hand:

1. Each dev-kit only covers a subset of all supported architectures.
2. Even within a given dev-kit, the included libraries do not necessarily provide the very same input/output APIs across the entirety of its supported targets.

Cross-Lib improves on these points and lets you do things that you cannot do with other tools:

1. It covers many more vintage architectures, including very exotic ones;
2. It lets you write 100% WORA code, i.e., "write once, run anywhere" (after re-compilation);
3. It has WORA graphics assets, i.e., define abstract graphics assets that are shared by several totally different systems;
4. It provides a simplified tool-chain to create, compile and test the games.

WHAT CROSS-LIB IS NOT

Cross-Lib is **not** similar to Infoc build tools [cate]. Infoc's games [cate2] have many versions and each version has its own source code.

The closest dev-kits to Cross-Lib with respect to providing a WORA hardware abstraction layer are:

- 8-bit-Unity [un8] (meant to be used with CC65), which does incredible things but only for very few MOS 6502-based systems;
- Z88DK [z88dk], which has huge libraries but only covers Zilog 80 and Intel 8080 systems.

On the other hand Cross-Lib is not a compiler. It uses several compilers to build the binary files (executable files, disk images, tape images, cartridge images, etc.) for retro-consoles and retro-computers.

By providing an abstraction layer for all vintage systems, Cross-Lib makes some compromises with respect to what we could do if we were coding only for a specific system. In particular, graphics is somehow limited by what is possible on all systems. Sound effects are even more limited.

COMPILERS

Cross-Lib is supposed to be used with C compilers but which ones?

A native compiler (for example GCC) can be used to build native binaries that can be run, for instance, under Windows. Today the support for native compilation is limited to the text terminal, which means no graphics other than ASCII on the host machine.

Currently Cross-Lib fully supports the following cross-compilers and dev-kits with graphics and sound effects for most of its targets:

- ACK [ack] for 8088/8086, Intel 8080 CP/M, 386/68K/PPC/MIPS Linux and PDP11,
- VBCC [vbcc] for the BBC Micro, BBC Master, the Amiga and others,
- CC65 [cc65] for most systems based on the MOS 6502 architecture;
- Z88DK [z88dk] for systems based on the Zilog 80 and Intel 8080 architectures;
- CMOC [cmoc] for systems that use the Motorola 6809;
- LCC1802 [lcc1802] for systems that use the COSMAC RCA 1802 ;
- GCC for TI [ti] for the Texas Instruments TI99/4A based on the 16-bit TMS9900.

Cross-Lib also supports other compilers (currently with limited or absent graphics and sound effects) such as

- XTC68 [xtc68] for the Sinclair QL,
- CC6303 [cc6303] for the Motorola 6803;
- several versions of modded GCC to target vintage computers (Atari ST [st], Olivetti M20 [m20]).

Cross-Lib supports a large subset of ANSI C89 because most available cross-compilers for vintage architectures only implement a subset of ANSI C89. Therefore we write in C (a subset of ANSI C C89) with Cross-Lib APIs for graphics, sounds, input. In particular we must avoid:

- "float" and "double" types,
- copies and parameters by value of "struct" objects,
- any use of the heap.

For hints on how to code efficiently for 8-bit systems we refer to [effc] and for specific hints on how to code with these compilers we refer to [c8b].

(NO) INSTALLATION

Cross-Lib itself does not require any installation.

An actual installation is not necessary because Cross-Lib is made of scripts and a C source library, which is compiled on the fly when building binaries for any target system.

For example, it is enough to download the repository files (from the GitHub page) or to clone the repository with:

```
git clone https://www.github.com/Fabrizio-Caruso/CROSS-LIB/
```

DEPENDENCIES

On the other hand it has the following dependencies:

- a POSIX environment (e.g., Windows with Cygwin, Linux, FreeBSD, etc.);
- Make (GNU Make);
- Python 2.7 or Python 3.*;
- GCC compiler;
- ncurses libraries (for the native terminal);
- any compiler necessary to build for a desired target architecture (e.g., CC65, Z88DK, etc.)
- JAVA (only to be disk images for the Apple //).

DOCKER

You don't need to install these dependencies if you use the docker image which contains most cross-compilers. For example you can run the following command:

```
docker run -it -v {cross-lib_location}:/usr/src/cross-lib ghcr.io/cross-lib/devcontainers/base:1.0.5-ubuntu
```

which will pull and run the docker container preloaded with Cross-Lib and its dependencies.

GITHUB CODESPACES

You can even run a Cross-Lib virtual environment on the browser without no need to use docker by using GitHub CodeSpaces. To use CodeSpaces with Cross-Lib, it is enough to fork the repository and launch CodeSpaces.

GET STARTED

Once you have Cross-Lib with its dependencies in a POSIX environment, you should move to the `src` directory. Inside `src` you will find the `x1` Python script and the

following directories

- "cross-lib", which contains the library code;
- "games" with built-in games;
- "examples" with built-in examples;
- "projects" with user-defined projects.

If you have correctly installed just Makefile, Python and any compiler (e.g. GCC), it should already be possible to minimally use Cross-Lib to build games and examples.

For the practical reasons I advice to have . in your PATH.

In order to test Cross-Lib from the src directory you can run x1 test .

THE XL SCRIPT

If just one of the supported compilers is installed, Cross-Lib can be used with the x1 Python script inside the "src" directory. The build command is

```
x1 build \<project name\> \<target name\> or just x1 \<project name\> \<target name\>
```

STDIO-ONLY BUILD

If you provide no target, x1 assumes you want to build the game for the stdio target, i.e., by only using the C standard library. This is useful only for testing or for turn-based games because the C standard library does not provide input/output functions other than buffered commands to print on the screen and read return-terminated lines from keyboard. This target only requires the GCC native compiler and does not need ncurses .

NATIVE BUILD

If GCC with ncurses is installed, a native version of the games and examples can be built with:

```
x1 build \<project name\> ascii or just x1 \<project name\> ascii
```

For example

```
x1 snake ascii
```

builds the "snake" game for the native console.

CROSS-BUILD

If a cross compiler (CC65, Z88DK, CMOC, CC6303, LCC1802, etc) is installed, the games and examples can be built for the corresponding systems.

For example, if CC65 is installed,

```
x1 horde apple2
```

builds the "horde" game for the Apple // with 6-color HGR graphics.

```
x1 snake oric
```

builds the "snake" game for the Tangerine Oric 1 or Oric Atmos in (simulated) color mode.

GETTING HELP ON XL

The x1 script takes several commands. In particular you can see a list of the available commands with x1 help .

You can get an explanation of each command with examples with

```
x1 help \<command\> .
```

CREATING A NEW GAME

If we want to create a new project, we need to use

```
x1 create \<project name\> ,
```

which will create a project with a trivial "hello world" example inside the "projects" directory.

We can also start with a template code for a game if we pass the additional parameter "game":

```
x1 create \<project name\> game .
```

HELLO WORLD

Coding with Cross-Lib is like normal coding in C (with ANSI C89) with the only addition that all input/output has to be done through Cross-Lib APIs.

Cross-Lib comes with many examples in the src/examples/ directory as well as a few games in the src/games directory. Both of which can be used to learn how to code with Cross-Lib. We could start to look at src/examples/hello/ , whose main.c files contains:

```
#include "cross_lib.h"

int main(void)
{
    _XL_INIT_GRAPHICS();
    _XL_CLEAR_SCREEN();
    _XL_SET_TEXT_COLOR(_XL_WHITE);
    _XL_PRINT(XSize/2-5,YSize/2,"HELLO WORLD");

    _XL_REFRESH();

    while(1){};
    return EXIT_SUCCESS;
}
```

We remark that all Cross-Lib specific functions start with the *XL* prefix:

1. `_XL_INIT_GRAPHICS()` required to initialize the display ;
2. `_XL_CLEAR_SCREEN()` which cleans the content of the screen ;
3. `_XL_SET_TEXT_COLOR(_XL_WHITE)` which defines the color to be used by the text and characters ;
4. `_XL_PRINT(XSize/2-5,YSize/2,"HELLO WORLD")` which displays "HELLO WORLD" on the center of the screen.
5. `_XL_REFRESH()` which actually refreshes the content of the screen and which is only useful when displaying in buffered mode (e.g., when only using `stdio.h`) and does nothing for most targets

If we want to use one of the examples or games as a starting point we can clone the project with:

```
x1 clone \<existing project to clone> \<new project name>
```

Or we can start from scratch with

```
x1 create \<new project name>
```

Once we have a new project, we can build it for the native termianl in ASCII mode with:

```
x1 \<project> ascii
```

if we want to build the project for the native terminal.

We can run the project for the native terminal with:

```
x1 run \<project>
```

For example we could run these commands:

```
x1 clone hello myhello [to clone hello and create myhellworld]
```

We modify the project myhello (found inside `./src/projects/myhello/`)

```
x1 build myhello ascii (or equivalently x1 myhello ascii ) [to build a native binary of the new project]
```

```
x1 run myhello [to run the new project]
```

```
x1 build myhello oric [to build the same project for the Oric]
```

CODING WITH ABSTRACTIONS

In order to code with Cross-Lib for all targets at the same time, Cross-Lib uses several abstractions.

Graphics is only tile-based. In the code, tiles are abstract as their exact shape will depend on the target type. The game code only sees tiles as macros `_TILE_0`, `_TILE_1`, `_TILE_2`, etc.

Each concrete target will interpret tiles in different ways depending on whether the target can or cannot assign free shapes to tiles and on the number of pixels that define tiles on that system.

Sound is currently only limited to few sound effects that are rendered somehow similarly across multiple targets.

Input is limited to 4 directions and to a single fire button. On each target this will be mapped to either 5 keys (usually `I` `J` `K` `L` `SPACE`) or the main joystick/controller/gamepad.

Running `x1 manual` will output a manual of Cross-Lib APIs.

MAIN DEVELOPMENT FLOW

Producing "universal" games with Cross-Lib for the developer boils down to these main steps:

1. Creating or cloning a project (with `x1 create` or `x1 clone`);
2. Writing the abstract game code in ANSI C with Cross-Lib APIs for input/output;
3. Manually creating or automatically importing abstract assets (with commands such as e.g., `x1 shapes`, `x1 import`, `x1 rip`, `x1 tile`);
4. Running the `x1` script to build the project for one or multiple targets (with `x1 build`).

The `x1 build <project> <target>` command (or its equivalent short form `x1 <project> <target>`)

executes one or several Makefile targets, which generally:

- generate a concrete asset file from the abstract asset files
- compile the source code (game code and target-specific portions of Cross-Lib library);
- include or link the asset files with the compiled source code;
- if necessary a tool is used to create a usable disk/floppy/rom/cartridge image (this is the case for the Apple II for which disk images are generated).

[INSERT FLOW PICTURE HERE]

CODING AND TESTING ALL TARGETS ON THE HOST TERMINAL

Cross-Lib lets you write the game code in a way that can be used to build binaries for more than 200 targets and possible several variations for each target. How are we supposed to test and debug the (abstract) source code for so many targets, without the need to test it on hundreds of targets through hundreds of emulators? We can do this by using the native terminal targets:

- `ascii` (aka "native" ASCII target)
- `terminal` (aka "native" graphics target)

Both such targets produce binaries that can be run on the host terminal.

THE ASCII TARGET

This target can **mimick any ASCII target**.

It represents an ASCII-only version for host terminal with default size 80x24. For example:

```
x1 verbix ascii
```

builds `verbix` for the host terminal and tiles are just ASCII characters.

Different screen sizes are possible by passing the x and y size of the screen in terms of number of tiles:

```
x1 horde ascii 20 18
```

builds `horde` with a 20x18 screen.

THE TERMINAL TARGET

This target is a graphics version for the host terminal, which can **mimick any target with graphics**.

By default it produces binaries with 20x17 as default screen size and 8x8 tile size.

For example:

```
x1 bomber terminal
```

builds `bomber` for the host terminal with a 20x17 screen and with 8x8 tiles.

This target (ab)-uses the terminal by using multiple full square characters as single pixels. So, in order to see the result, you need to resize the font to its minimal size.

Change screen size

We can use different screen sizes can be set by passing them as arguments.

For example:

```
x1 shoot terminal 24 20
```

builds `shoot` with a 24x20 screen.

Change tile size

We can also change the tile size by using the following alternative terminal targets:

- `terminal16x8`
- `terminal16x9`
- `terminal18x6`
- `terminal17x8`

For example:

```
x1 stinger terminal18x6
```

builds `stinger` with 8x6 tiles.

Mimick a target

For a large number of targets we can just use `terminal` followed by the name of the target we want to mimick (same tile and screen size).

For example:

```
x1 snake terminal apple2
```

builds `snake` with the same screen and tile sizes as the Apple //, i.e., with a 20x24 screen and with 7x8 tiles.

GRAPHICS

Graphics is performed by displaying "tiles", whose shape can be defined. We can also display "characters" (letters, digits and the space character) but their shape is (currently) fixed. Characters and tiles have the same size. Characters are used for the text and digits, whereas tiles are used for graphics.

With respect to graphics, we have two types of targets:

- ASCII-only targets for which we can only assign ASCII characters to tiles;
- targets with real graphics we can define the shapes of the tiles.

Currently each tile only has one tile-specific foreground color and a common background color. The definition of their shapes is not done in the C code. Instead it is done inside the `<project>/tiles` directory, which contains sub-directories with files with tile definitions for the ASCII case and for the graphics-enabled targets, e.g., 8x8 (for most targets), 7x8 (for the Apple //), 6x8 (for the Oric 1, Oric Atmos, PMD-85, NTSC COMX-35), etc...

In the aforementioned files the shapes are described by either ASCII codes for ASCII-only targets or sequences of bytes for graphics-enabled targets.

These files can be:

1. edited manually or
2. automatically imported from several programs such as CharPad [chPad] (with `x1 import \<file> \<project>`) or
3. automatically ripped from BASIC or Assembly listings (with `x1 rip \<file> \<project>`) or
4. generated by some simple "pictorial" text files with drawings made of "#" and "." characters

(with `x1 shapes \<project> \<x_size> \<y_size>` or `x1 tile \<file> \<project> \<tile number>`).

You can see how to use these commands by running `x1 help \<command>`.

In the game code tile graphics is primarily performed by invoking these functions (to display or delete tiles):

```
void _XL_DRAW_TILE(uint8_t x, uint8_t y, uint8_t tile_number, uint8_t tile_color);

void _XL_DELETE(uint8_t x, uint8_t y);
```

SOUND

Sound is even more limited than graphics and it is restricted to a few predefined effects.

The sound commands have this form:

```
void _XL_<effect name>_SOUND(void);
```

where the currently possible effects are `EXPLOSION`, `SHOOT`, `TICK`, `TOCK`, `PING`, `ZAP`.

For example `XL_EXPLOSION_SOUND()` produces an explosion sound or something vaguely similar to it depending on the target.

INPUT

Input is limited to four directions and a single fire button.

The primary input APIs are:

```
uint8_t _XL_INPUT(void);

uint8_t _XL_LEFT(uint8_t input);

uint8_t _XL_RIGHT(uint8_t input);

uint8_t _XL_UP(uint8_t input);

uint8_t _XL_DOWN(uint8_t input);

uint8_t _XL_FIRE(uint8_t input);
```

where `_XL_INPUT` polls the input and the other functions are used to determined the specific input.

GRAPHICS, SOUNDS AND INPUT IN ONE EXAMPLE

Inside the `src/examples/animate` directory you can find the following example that could be used as a blueprint for a simple game.

This simple example has graphics, input and sound.

When compiled for any of the possible systems, it will produce the very same program but the exact tile shapes and sounds will depend on the target.

For example it can be compiled for the native console with:

```
x1 animate ascii
```

or for the Commodore Vic 20 with

```
x1 animate vic20
```

or for the MSX with

```
x1 animate msx
```

ORIC SUPPORT

Recently I have greatly improved the support for the Oric 1 and Oric Atmos machines with 3 new video modes:

- `oric_mono` target: high resolution monochrome mode with 6x8 tiles in a 40x25 screen
- `oric` target: quad pixel resolution with color 6x8 tiles in a 19x12 or 19x13 screen (depending on game configuration)
- `oric_39x25` target: high resolution mode with limited color support

Monochrome mode support

The `oric_mono` target will use hardware text mode in Cross-Lib and hardware high resolution mode in Cross-Lib-HD (in order to provide 128 total tiles).

For example we can build `trex` for the Oric in monochromatic mode with: `x1 trex oric_mono`.

[[trex_oric_mono image](#)]

Standard color mode

The `oric` target uses hardware high resolution mode and provides 64 or 128 total tiles with colors. Colors are currently produced by having alternating cyan and yellow attributes. Inversion is performed on fonts to render red and white colors. Inversion on tiles is possible if the `__INVERSE_TILES` is set in the `ORIC_COLOR_OPTS` variable in `project_config.mk`. Pixels have quadruple size compared to hardware high resolution and only 19x12 tiles are displayed on the screen. If `Force_YSize=13` is set in `ORIC_COLOR_OPTS` then an extra monochromatic half-height line is displayed on the bottom of the screen.

For example we can build `snake` for the Oric in standard color mode with `x1 snake oric`.

[[snake_oric image](#)]

High-resolution mode with limited color support

The `oric_39x25` (alias `oric_alt`) target uses the high resolution mode and provides 64 or 128 total tiles with limited colors. We do define alternating cyan and yellow attributes but each line of each tile is either on a cyan line or a yellow line. We cannot just display even or odd lines without losing half the details of our tiles. So we are forced to use color inversion more heavily if we want to get more colors.

Colors are enabled by default on fonts. They be disabled if `__MONO_TEXT` is set in `ORIC_39x25_COLOR_OPTS`. Colors can be enabled on tiles if the `__INVERSE_TILES` is set in the `ORIC_39x25_COLOR_OPTS` variable in `project_config.mk`. Colors on such small fonts and tiles may not look great because they

rely on color inversion. This is why Cross-Lib provides ways to disable colors.

For example we can build `snake` for the Oric in high-resolution color mode with `x1 snake oric`.

[snake_oric_39x25]

CROSS-LIB GAMES (May 2026)

The repository of Cross-Lib contains some games I have written with it. As for June 2026, it includes nine games:

- **Trex**: jump to avoid cacti and birds
- **Stinger**: tank-themed shoot'em-up
- **Shuriken**: Pacman meets Pengo sort of game
- **Verbix**: Scrabble meets Tetris sort of game
- **Horde**: zombie-themed shoot'em-up
- **Snake**: Nibbler meets Nokia Snake
- **Bomber**: clone of Air Attack
- **Shoot**: Robotron-like shooter
- **Chase**: real-time Robots/Daleks

All current games have an ending. For some games it is reasonably difficult to complete the game. For some other games, it is much harder.

TREX

TREX is an obvious clone of Google Chrome's Easter egg mini-game. Avoid cacti and birds. Use SPACE or the fire button to jump to . TREX has 99 (!!!) levels.

See section above for a screen shot of an Oric version.

STINGER

Stinger a tank-themed shooter with several power-us and different enemies, levels and items. Some items are secret and hard to get.

You control a stinger with a limited number of missiles. You can get more missiles as a possible reward for destroying some tanks.

More numerous and harder tank types appear as you progress to higher levels. The game has 6 levels.

The image below shows the version for the Commodore VIC 20.

[stinger_vic20 image]

SHURIKEN

Shuriken is somehow similar to PAC-MAN in that you need to collect all diamonds to complete non-boss levels while you need to avoid being hit by the shurikens. In boss levels you have to additionally kill all shurikens by using the movable blocks that are deadly for them. You can also collect special items that make you temporarily invincible (white ring) or that slow down the shurikens (blue large diamond). The game has 12 levels with 3 boss levels.

The image below shows the version for the Commodore VIC 20.

[shuriken_vic20 image]

VERBIX

Verbix is a Scrabble-like game. In this game you have to form valid 5-letter English words on the bottom row before the tower of letters reaches the top level. You can rotate the bottom row or any of the columns by moving in the 4 directions. You press fire once you have found a valid word. The game has 9 levels.

The image below shows the version for the MSX.

[verbix_msx image]

HORDE

Horde is a zombie-themed shooter with many power-ups and several different enemies, levels and special items to pick.

You control a bow with a limited number of arrows. You get more arrows as a possible reward for killing some zombies. If you run out of arrows, you will get a few arrows after a short period. In each level you are confronted with a horde of zombies that become harder as you progress. The game has 9 levels.

The image below shows the version for the Thomson M05.

[horde_mo5 image]

SNAKE

Snake is my personal interpretation of a hybrid game between the arcade game Nibbler and the Nokia cell phone game Snake. As in Nibbler the walls do not kill you and can be used to stop you and let you better control your movements. As in Nokia Snake the main goal is to eat apples. If you do not eat any apple for too long, the snake becomes faster and harder to control, consumes its energy and eventually dies. In each level you are also confronted with mines that kill you if the snake's head touches them. As a secondary goal you can collect rings. If you manage to get 3 or more rings without dying on a level, you will be rewarded with bonus items.

The game has many secrets to discover. It has a secret level and 32 normal levels including 8 bonus levels.

The section above has screenshots of two Oric versions of the game.

BOMBER

Bomber is a mini-game and clone of Air Attack (aka Blitz) that first appeared on the Commodore Pet. The game concept is horrible (not my own concept) but it can be addictive nevertheless. Your plane has to carpet-bomb an entire city in order to safely land on the road. Let us assume the city has no inhabitants. You do not control the plane and you can only press fire to drop a bomb at a time. The game has 9 levels.

The image below shows a colored version for the Oric.

[bomber_oric image]

SHOOT

Shoot is a shooter somehow similar to Robotron. You have to kill a horde of ghosts on each level. You can kill them by hitting them many times or by hitting them enough times to push them against the walls (mortal only for them). You can also kill them by placing mines but you need to get the mine item (with 3 mines). A mine kills instantly common enemies. At the end of each level when only few ghosts are left or if you survive without killing enemies long enough or in boss levels, you are confronted by some hard-to-kill skull-shaped enemies. As a secondary goal you can complete the level by killing the skulls before the last ghosts and by doing so you unlock extra items and secrets in the next levels. Destroying the missile bases can also unlock items. The game has many secrets to discover. It has a secret level, 16 normal levels including 2 boss levels.

The image below shows the version for the Apple //.

[shoot_apple2 image]

CHASE

Cross Chase is somehow similar to Daleks / Gnome Robots. You need to lure your pursuers into the mines. The main difference is that it is a real-time game and that it has several secrets items and power-ups to pick. Similarly to Shoot, at the end of each level or in boss levels, you are confronted by a boss (a skull), which can only be killed by several bullets fired by the gun item. You can complete the level by killing the skull before the last common enemy and by doing so you unlock extra items and secrets in the next levels. Destroying the missile bases can also unlock items. The game has many secrets to discover. It has 20 normal levels including 4 boss levels.

The image below shows the version for the Oric.

[chase_oric]

FUTURE DEVELOPMENT

Cross-Lib is actively developed. Planned future developments are:

- support for more targets and architectures (e.g., Apple //gs, Apple 1, Super NES, etc.);
- improved targets (e.g. DHGR mode for the 80-column capable Apple // models);
- improved APIs (more tiles, improved sounds, etc.);
- simplified setup (simpler way to use the docker image);
- improved support for native targets;
- more games.

BIBLIOGRAPHY

[ack] ACK, Tanenbaum A., Jacobs C. et al., C compiler for Intel 8088, PDP 11, CP/M-80, etc., <https://github.com/davidgiven/ack>

[app] Cross Chase: A Massively 8-bit Multi-System Game, Caruso F., Call-A.P.P.L.E., vol. 28, No. 1, Feb 2018, pages 31-33, <http://www.callapple.org>

[app2] Introducing Cross-Lib, Caruso F., Call-A.P.P.L.E., vol 3, No. 1, Jan 2023, <http://www.callapple.org>

[cate] Inufuto Cate compiler, <https://github.com/inufuto/Cate>

[cate2] Inufuto Cate games, https://github.com/inufuto/Cate_examples

[c8b] 8-bit C, Caruso F., <https://github.com/Fabrizio-Caruso/8bitC>

[cc6303] CC6303, Cox A., C compiler for the Motorola 6800 and 6803, <https://github.com/EtchedPixels/CC6303>

[cc65] CC65, C compiler for the MOS 6502, <https://sourceforge.net/projects/cc65/>

[ceo] Cross Chase, Caruso F., CEO-MAG, No. 327-328, Juillet-Août 2017, <https://ceo.oric.org/>

[chPad] CharPad, Subchrist Software, <https://subchristsoftware.itch.io/charpad-c64-free>

[cmoc] CMOC, Sarrazin P., C compiler for the Motorola 6809, <https://perso.b2b2c.ca/~sarrazip/dev/cmoc.html>

[effc] Efficient C Code for 8-bit Microcontrollers, Jones N., <https://barrgroup.com/embedded-systems/how-to/efficient-c-code>

[m20] Z8KGCC, Groessler C., GCC modded for the Olivetti M20, <http://www.z80ne.com/m20/sections/download/z8kgcc/z8kgcc.html>

[lcc1802] LCC1802, Rowe B., C compiler for the RCA 1802, <https://github.com/bill2009/lcc1802>

[prog] Cross-Lib, Caruso F., Programmez! Hors Série #6, pages 72-82.

[rg] Interview to F. Caruso about Cross-Lib, RetroGamer #240, page 100, <https://www.retrogamer.net/>

[st] GCC, Rivière V., modded for the Atari ST, <http://vincent.riviere.free.fr/soft/m68k-atari-mint/>

[ti] GCC, (Insomnia), modded for the TI99/4A, <http://atariage.com/forums/topic/164295-gcc-for-the-ti>

[un8] 8-Bit Unity, Beaucamp A. (8-bit Dude), <http://8bit-unity.com/>

[vbcc] VBCC, Barthelmann V., C compiler for the BBC Micro, BBC Master, Amiga, etc., <http://www.compilers.de/vbcc.html>

[xtc68] XTC68, Hudson J., C compiler Sinclair QL, <https://github.com/stronnag/xtc68>

[z88dk] Z88DK, C compiler and dev-kit for the Zilog 80 and Intel 8080 architectures.